



GUÍA DIDACTICA ACTIVIDAD NRO 2

Desarrollo de aplicación de escritorio cliente
servidor con acceso a datos Herramientas de
programación III

Descripción breve

Guía de tecnologías aplicadas para acceso a datos desde aplicaciones de escritorio.

Javier Saldarriaga Cano
Javier.saldarriaga@pascualbravo.edu.co



ACCESO A BASE DE DATOS CLIENTE SERVIDOR

OBJETIVO: Conocer y practicar por medio de la actividad, las estrategias y técnicas de acceso a bases de datos utilizando modelo de desarrollo cliente-servidor con C# y utilizando el motor de base de datos Microsoft SQL SERVER.

Introducción

Una base de datos es un “almacén” que nos permite guardar grandes cantidades de información de forma organizada para que luego podamos encontrar y utilizar fácilmente. Cada base de datos se compone de una o más tablas que guarda un conjunto de datos. Cada tabla tiene una o más columnas y filas. Las columnas guardan una parte de la información sobre cada elemento que queramos guardar en la tabla, cada fila de la tabla conforma un registro.

Se define una base de datos como una serie de datos organizados y relacionados entre sí, los cuales son recolectados y explotados por los sistemas de información de una empresa o negocio en particular. Las Bases de Datos tienen una gran relevancia a nivel personal, pero más si cabe, a nivel empresarial, y se consideran una de las mayores aportaciones que ha dado la informática a las empresas.

En la actualidad, cualquier organización que se precie, por pequeña que sea, debe contar con una Base de Datos, pero para que sea todo lo efectiva que debe, no basta con tenerla: hay que saber cómo gestionarlas.

En esta nueva actividad nos adentraremos en el acceso y manejo de bases de datos desde C#, para esto veremos que técnicas ofrece Microsoft C# para acceder a los datos, que sentencias se requieren para manejo de la información obtenida de una base de datos y como administraremos esta información por medio de Windows Form.

Motor de base de datos

En el desarrollo de las actividades propuestas en este curso, podrán utilizar indistintamente los motores de bases de datos MariaDB o SQL Server, de acuerdo con su preferencia, experiencia previa o disponibilidad del entorno de trabajo. Esta flexibilidad busca fortalecer la comprensión de los conceptos fundamentales de bases de datos, promoviendo el análisis, diseño y manipulación de la información de manera independiente de la tecnología específica utilizada. De esta forma, se fomenta una formación práctica y adaptable a diversos contextos profesionales y tecnológicos.



Uso de MariaDB en entorno local con XAMPP

Para desarrollar las actividades del curso, deberás trabajar con el motor de base de datos MariaDB de forma local en tu computador utilizando la herramienta XAMPP. Esta herramienta te permite contar con un entorno de trabajo completo que incluye el servidor de base de datos necesario para realizar los ejercicios y proyectos propuestos.

Como primer paso, descarga e instala XAMPP desde su sitio oficial, seleccionando la versión correspondiente a tu sistema operativo. Una vez finalizada la instalación, abre el Panel de Control de XAMPP y activa el servicio MySQL; este servicio corresponde al motor MariaDB incluido en XAMPP.

Con el servicio en ejecución, accede desde tu navegador web a la dirección <http://localhost/phpmyadmin>. A través de esta interfaz podrás crear bases de datos, tablas y usuarios, así como ejecutar las sentencias SQL requeridas en cada actividad del curso.

Ten en cuenta que, por defecto, MariaDB en XAMPP se ejecuta con los siguientes datos de conexión:

- **Servidor:** localhost
- **Puerto:** 3306
- **Usuario:** root
- **Contraseña:** (vacía)

Estos datos podrás utilizarlos para conectar tus aplicaciones o herramientas de desarrollo a la base de datos, según lo indicado en cada ejercicio.

Trabajar con MariaDB de forma local te permitirá practicar de manera segura, realizar pruebas sin riesgo y fortalecer tus conocimientos en bases de datos, facilitando tu proceso de aprendizaje a lo largo del curso.

Descargar Aquí :el [script de la base de datos para Mysql/Mariadb](#)

Instalación SQL Server

Es importante saber que un servidor de bases de datos relacionales es un sistema bajo arquitectura cliente/servidor que proporciona servicios de gestión, administración y protección de la información (datos) a través de conexiones de red, gobernada por unos protocolos definidos y a los que acceden los usuarios, de modo concurrente, a través de aplicaciones clientes.

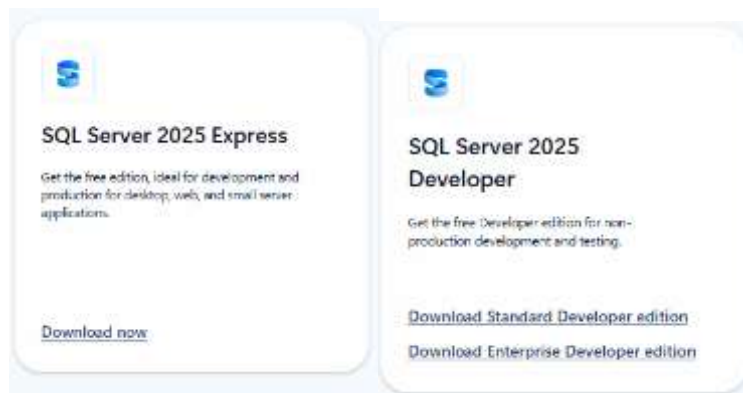


SQL Server es un sistema de gestión de bases de datos relacionales (**RDBMS**) de Microsoft que está diseñado para el entorno empresarial. SQL Server se ejecuta en T-SQL (Transact -SQL), un conjunto de extensiones de programación que añaden varias características a SQL estándar, incluyendo control de transacciones, excepción y manejo de errores, procesamiento fila, así como variables declaradas.

Para instalar Sql Server nos dirigimos a la página oficial del producto al área de descargas:

<https://www.microsoft.com/en-us/sql-server/sql-server-downloads>

Aquí se dispone de diferentes ediciones del producto, para nuestro trabajo utilizaremos **SQL Server 2025 Express**, o **SQL Server 2025 Developer** las cuales son ediciones gratuitas de SQL Server ideal para el desarrollo y la producción de aplicaciones de escritorio, aplicaciones web y pequeñas aplicaciones de servidor.



Se descarga el siguiente archivo **SQL2025-SSEI-Expr.exe** con este procederemos a la instalación, Para ver en detalle las indicaciones de cómo instalar el producto te recomiendo ver el siguiente

Video: https://youtu.be/SqLpurhoMkA?si=mf-m7Xfdk_uAbVB3

Restaurar la base de datos “DBFACTURAS”

Para nuestro proyecto utilizaremos la base de datos llamada “**DBFACTURAS**” la cual contiene las tablas y procedimientos almacenados necesarios para nuestra aplicación. Con el fin de ahorrar tiempo y facilitar el inicio del desarrollo del aplicativo, he compartido un Backup y el script de la base de datos para Microsoft sqlserver : los cuales podrá descargar desde los siguientes links



script base de datos : [Descarga Archivo Script](#)

BACKUP_BD_FACTURAS : [Descarga Archivo Backup](#)

Se recomienda inicialmente utilizar el backup para restaurar, en caso de tener problemas proceda a utilizar el script, un buen video que le orientara tanto en la creación como en la restauración del backup lo encuentra en: <https://youtu.be/svpQow0wOcl>

Proyecto de acceso a bases de datos Cliente Servidor en C#

Para desarrollar la actividad partiremos de lo realizado en la “**AEAE 2:Diseñando la Interfaz gráfica en aplicaciones de escritorio**”. Utilizaremos esta interfaz gráfica desarrollada y sobre esta implementaremos la funcionalidad de acceso a la base de datos.

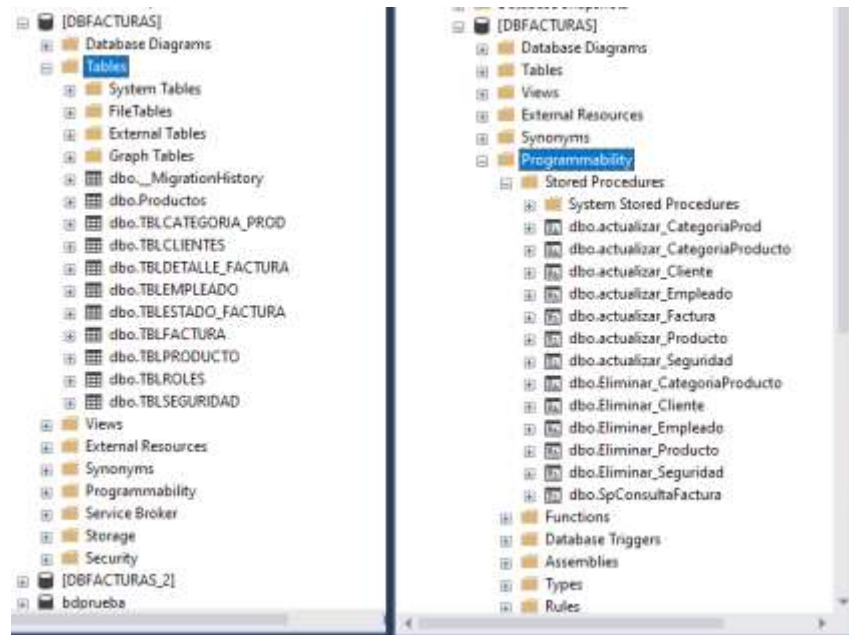
IMPORTANTE: Se recomienda primero crear una copia del proyecto “**Proyecto_Sistema_Facturacion**” para no alterar el original y la copiamos en una carpeta llamada “**Facturacion_Cliente_Servidor**” Realizamos esta copia del proyecto debido a que la interfaz gráfica la utilizaremos de nuevo para la próxima actividad.

A continuación, se detalla paso a paso las actividades que realizaremos para iniciar el desarrollo de nuestro aplicativo de facturación.

Una vez restaurada la base de datos encontrara las tablas y procedimientos almacenados que se utilizara en las diversas actividades del curso, por esto se recomienda revisar una a una las tablas para que identifique los campos que las componen y los tipos. Así mismo se recomienda revisar cada uno de los procedimientos almacenados , se han creado dos procedimientos almacenados por cada tabla, el primero de ellos permite la actualización de información para insertar y actualizar registros y el segundo procedimiento se ha creado para realizar el borrado de registros, la siguiente imagen muestra las tablas y procedimientos de la



base de datos.



Recuerde que en el documento llamado **“ESTRUCTURA _TABLAS _BD__FACTURAS”** se describen estas tablas y los campos que la componen.

[Ver documento de estructuras de tablas](#)

O pega el siguiente link en tu navegador:

<https://docs.google.com/spreadsheets/d/1kQ0keYG-tsRxYyEXXeGN2BeHRzd1rZSe/edit?usp=sharing&ouid=115752736545562171331&rtpof=true&sd=true>



Creación de los formularios del programa de facturación.

Una vez tengamos creadas las tablas y los formularios procedemos en esta guía a desarrollar el código que nos ilustrará cómo podemos acceder y actualizar información en la base de datos, los siguientes serán los formularios que nos servirán de ejemplo para desarrollar las siguientes funcionalidades:

- **Validar el usuario,**
La validación de acceso al sistema se realiza desde el formulario de **frmLogin**, desde aquí accedemos a la base de datos y verificamos si el usuario está almacenado en las tablas **TBLSEGURIDAD** y en **TBLEMPLEADO**, si es así se permite el acceso al sistema mostrando el formulario **frmPrincipal** de lo contrario muestra el mensaje de **usuario clave no encontrados**
- La siguiente acción consistirá en programar el formulario **frmPrincipal** para que permita el llamado de los diversos formularios que utilizará el programa, este llamado lo realizamos desde las opciones de menú que se crearan (esta funcionalidad ya se ha realizado en la práctica anterior).
- **Administrar Cliente**, este tiene como función la presentación y la actualización de la información que se encuentra almacenada en la tabla **TBLCLIENTES**. esta funcionalidad se implementa en el formulario **frmClientes**
- **Administrar seguridad**, esta funcionalidad la realizaremos desde el formulario **FrmAdminSeguridad**, el cual permitirá consultar los empleados y facilitará asignarles un usuario y una clave para que puedan acceder al sistema y también permite el cambio de los valores de usuario y password para aquellos que ya estén registrados

Para realizar lo anteriormente descrito es necesario que se conozca las técnicas que Microsoft ofrece para acceder a los datos y manipularlos acorde a las funcionalidades que requerimos hacer en la base de datos, a continuación, detallamos estas tecnologías:

El acceso a datos en C#

En esta sección vamos a definir la manera de conectarnos con una base de datos para interactuar desde C#, para acceder a una base de datos utilizamos un conjunto de componentes denominados ADO.NET. Que es la sigla de ActiveX Data Objects

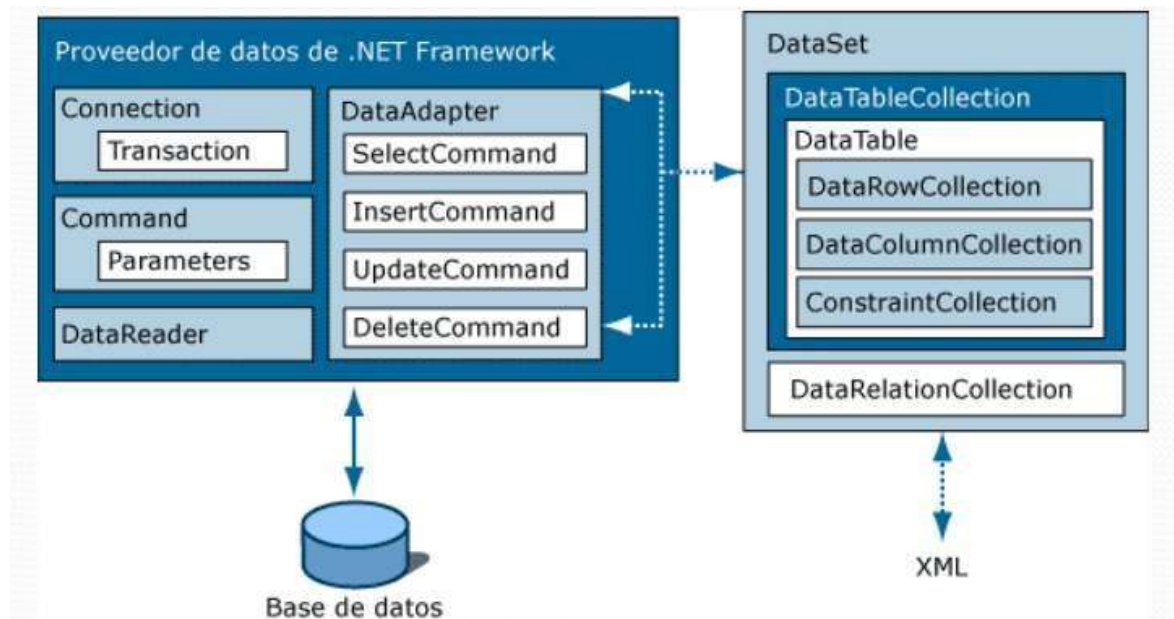


ADO.NET.

ADO.NET es un conjunto de componentes del software que pueden ser usados por los programadores para acceder a datos y a servicios de datos. Es parte de la biblioteca de clases base que están incluidas en el Microsoft .NET Framework. Es comúnmente usado por los programadores para acceder y para modificar los datos almacenados en un Sistema Gestor de Bases de Datos Relacionales como microsoft SQL, aunque también puede ser usado para acceder a datos en fuentes no relacionales. ADO.NET puede ser utilizado desde cualquier lenguaje de programación de .NET.

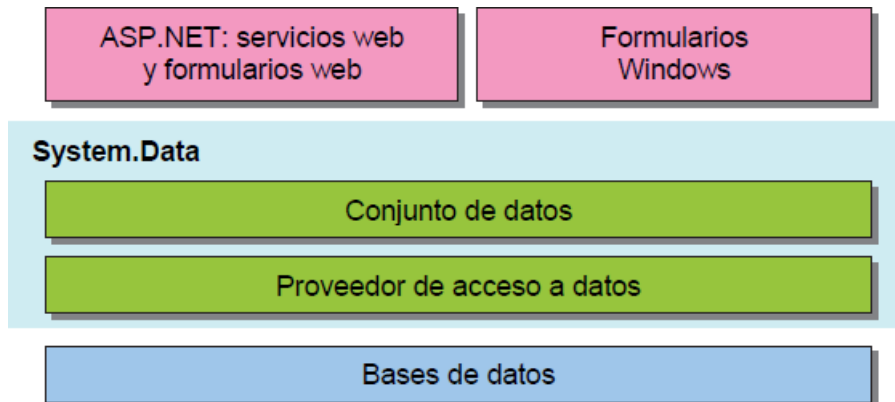
Los componentes de ADO.NET son los siguientes

- Connection (conexión)
- Command (Comandos, ordenes)
- DataReader (lector de datos)
- DataAdapter (adaptador de datos)



Como podemos ver en la imagen se puede decir que el trabajo de conexión con la base de datos, o la ejecución de una sentencia SQL determinada para recuperar datos de la base de datos, lo realiza el **"Proveedor de datos de .net"**.

En cambio, recuperar esos datos para tratarlos, manipularlos o volcarlos a un determinado control o dispositivo es una acción ejecutada por una capa superior formada por un **"Conjunto de Datos"** agrupado en tablas. Gráficamente lo anterior descrito se aprecia en la siguiente imagen



En .NET Framework, un proveedor de datos sirve como puente entre una aplicación y un origen de datos (**BASE DE DATOS**). Se utiliza tanto para recuperar datos de un origen como para actualizarlos. Los componentes principales de un proveedor de datos .NET son los objetos siguientes:

- ❖ **Conexión** con el origen de datos (objeto **Connection**). Establece una conexión a un origen de datos determinado.
- ❖ **Órdenes para acceso a los datos** (objeto **Command**). Ejecuta una orden SQL o un procedimiento almacenado en un origen de datos.
- ❖ **Lector de datos** (objeto **DataReader**). Proporciona una forma rápida de acceder a los datos recuperados después de una consulta a la base de datos. El acceso permitido es solo para leer y hacia delante.
- ❖ **Adaptador de datos (objeto DataAdapter)**. Llena un **DataSet** y realiza las actualizaciones necesarias en el origen de datos.

SqlConnection y La Conexión con la Base de Datos

SqlConnection es una clase que nos sirve para construir objetos de conexión a una base de datos de Sql Server. La clase de conexión SqlConnection provee funcionalidad a los objetos para establecer los parámetros necesarios para conectarse a una base de datos, estos parámetros se proveen a través de un constructor de la clase o bien asignándolos en la propiedad **ConnectionString**, como tal es una cadena de caracteres y donde se establecen los valores necesarios para que el objeto de conexión intente conectarse a la base de datos, a esta cadena se le conoce como **Cadena de Conexión**.



La cadena de conexión tendrá dos formas dependiendo del método de autenticación para la conexión, dado que SQL Server maneja dos tipos de autenticación; uno conocido como **autenticación integrada de Windows**(la que utilizaremos para este proyecto), que se caracteriza por la autenticación mediante el uso del usuario en sesión validado contra los usuarios de equipo o contra usuarios de un dominio.

El otro tipo es mediante la **autenticación con usuarios definidos en SQL Server** a la cual se le llama autenticación estándar, es por eso que se deben de poder definirse dos tipos de cadenas de conexión para cada caso en particular.

Una cadena de conexión con autenticación integrada de Windows que utilizaremos para nuestro proyecto se define de la siguiente manera:

“Server=NombreServidor; Database=NombreBaseDeDatos; Integrated Security=SSPI”

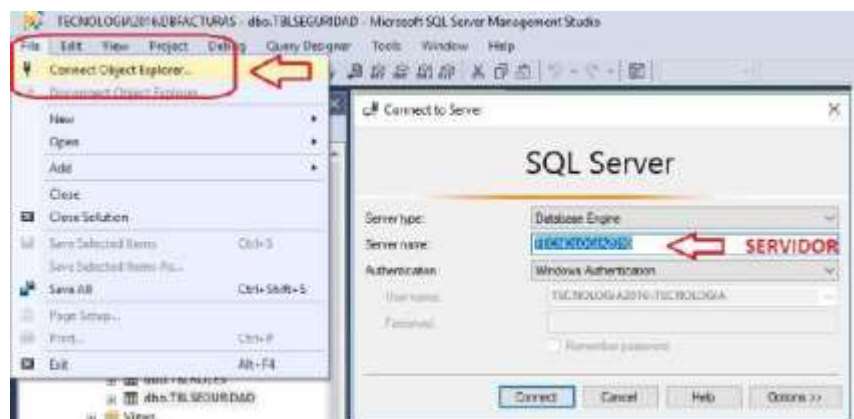
Ejemplo:

“Server= TECNOLOGIA2016; Database= DBFACTURAS; Integrated Security=SSPI”

La cadena de conexión con autenticación estándar se define de la siguiente forma:

“server=NombreServidor; database=NombreBaseDeDatos; User Id= UsuarioLogin; Password=MiContraseña”

Nombre del servidor: para identificar el nombre de nuestro servidor se debe Acceder al Microsoft SQL server y en el menú superior seleccione “File” y luego selecciona conectar a un servidor , se mostrará la siguiente pantalla:





SqlCommand y la Manipulación de Datos

La clase **SqlCommand** es una de las clases más utilizadas en ADO.NET junto con **SqlConnection**, la clase **SqlConnection** sirve para conectarse con la base de datos y **SqlCommand** se utiliza para manipular los datos del servidor utilizando un objeto de conexión construido con **SqlConnection**.

SqlCommand sirve para establecer las consultas e instrucciones SQL que se ejecutarán en el servidor y también para ejecutar procedimientos almacenados.

SqlCommand tiene dos propiedades principales, la primera es **Connection** que es donde se asigna el objeto de conexión construido con [SqlConnection](#) y la segunda es [CommandText](#), que es donde se asigna el texto de la instrucción SQL a ser ejecutada en el servidor.

En ocasiones será necesario asignar otras dos propiedades dependiendo del [CommandText](#) que se utilice, una de estas propiedades es [CommandType](#), que es donde se define si el [CommandText](#) es el nombre de un procedimiento almacenado, una sentencia SQL o si es el nombre de una tabla. La otra propiedad es [CommandTimeout](#) que sirve para asignar el tiempo de espera en segundos tras intentar ejecutar una instrucción definida en [CommandText](#), al término del tiempo de espera después de intentar ejecutar el [CommandText](#), se lanza una excepción. Las excepciones que ocurran en el servidor de **Sql Server** serán notificadas con todas sus características por medio de un objeto del tipo **SqlException**, en donde podremos encontrar el mensaje y número del error.

En el siguiente ejemplo realizaremos la conexión y consulta de la tabla de clientes de la base de datos.

Creamos un proyecto nuevo y le das como nombre “EjemploADO” y renombramos el primer formulario(form1) con el nombre **frmListaClientes**, le agregamos un título y un **DataGridView** y le asignamos el nombre **dgClientes** y contiene los siguientes títulos de columnas:



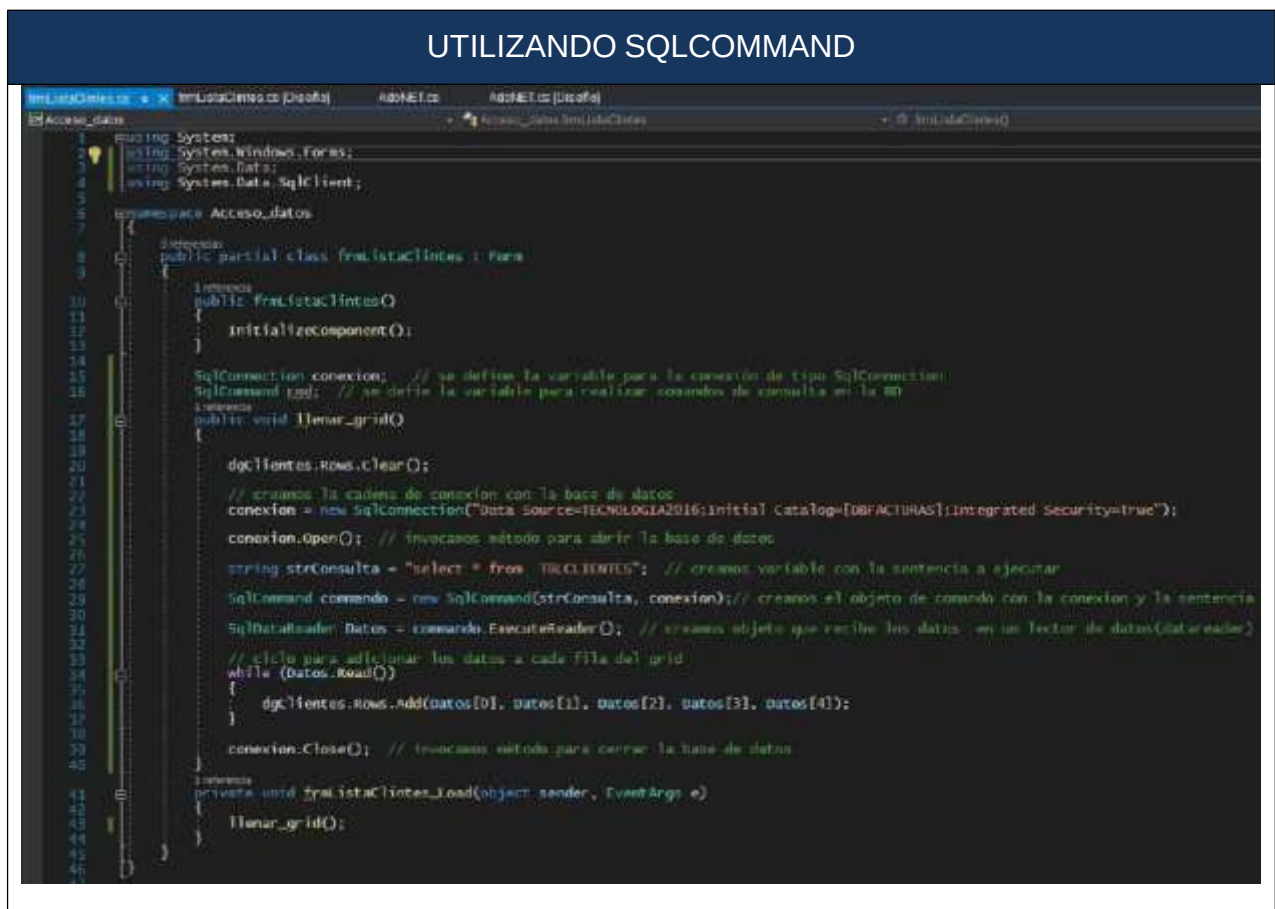


El siguiente ejemplo muestra cómo utilizar **SqlCommand** para ejecutar sentencias de consulta en la base de datos. Es necesario que se realice la vinculación de librerías para acceso a datos:

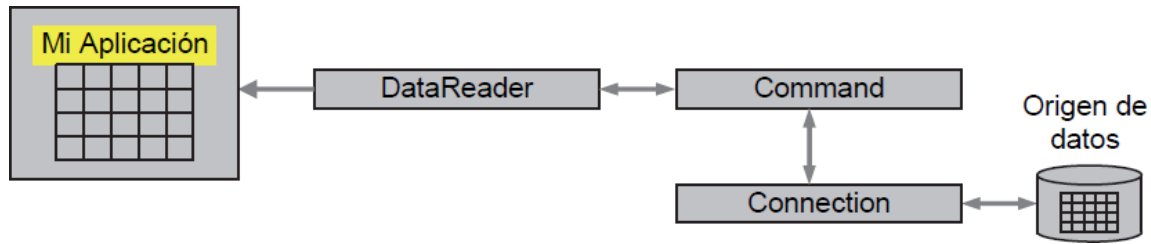
```
1 using System;  
2 using System.Windows.Forms;  
3 using System.Data;  
4 using System.Data.SqlClient;
```

Para indicar la ejecución de la instrucción SQL utilizaremos los métodos **ExecuteReader()** y **ExecuteNonQuery()**

ExecuteReader se utiliza para ejecutar una consulta SELECT y devuelve un objeto **DataReader**. En el siguiente ejemplo vemos como se utiliza:



Cuando una aplicación solo necesite leer datos (**NO ACTUALIZARLOS**), basta utilizar un **objeto lector de datos**. Un objeto lector de datos obtiene los datos del origen de datos y los pasa directamente a la aplicación, El objeto lector de datos proporcionado por .NET para SQL Server es **SqlDataReader**, La figura siguiente muestra cómo se utilizan este objeto:



La consulta la ejecutamos con el llamado al método **ExecuteReader()** el cual nos retorna un conjunto de registros los cuales almacenamos en un objeto que debemos definir como **SqlDataReader**.

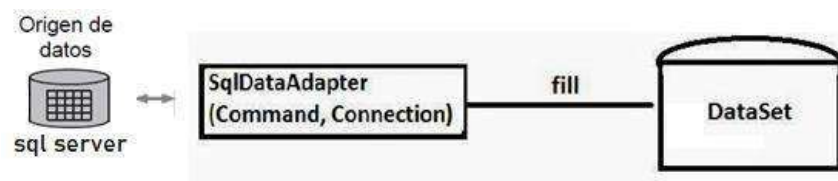
El método **Read** desplaza el cursor del DataReader, de solo lectura y avance hacia delante, hasta el siguiente registro. La posición inicial es antes del primer registro, por lo tanto, se debe llamar a Read para iniciar el acceso a los datos. Una vez finalizada la lectura se debe llamar al método Close() para liberar el Data- Reader de la conexión, objeto Connection.

CONSULTA Y ACTUALIZACION DE DATOS

SqlDataAdapter(Adaptador de datos)

El adaptador de datos permite la **consulta y actualización** de datos, un adaptador es un conjunto de objetos utilizado para intercambiar datos entre la base de datos y un conjunto de datos (objeto **DataSet**). Esto significa que una aplicación leerá información de una base de datos para un conjunto de datos(**DataSet**) y, a continuación, si lo requiere manipulará dichos datos para modificarlos y posteriormente actualizarlos en la base de datos

En la siguiente figura se puede observar la utilización de un adaptador(DataAdapter) de datos para llenar un conjunto de datos (objeto DataSet)



Un dataset, como su nombre indica, es un conjunto de datos, que habitualmente están estructurados como una matriz en filas y columnas, como ejemplo podríamos decir que una tabla de una base de datos de SQL sería un dataset, en el que cada fila corresponde a un registro que tiene toda la información de un ítem particular.



Finalmente, toda la información quedara almacenada en un objeto llamado **DataTable**, el DataTable representa una tabla de datos relacional en memoria; los datos son locales, pero luego estos pueden ser actualizados en la base de datos.

En el siguiente ejemplo realizaremos una consulta de una tabla utilizando un dataAdapter y llevándolo a un dataTable para mostrar en pantalla la información.

La siguiente es la pantalla creada para presentar información en un datagridview y en un combo, el botón nos permite identificar cuál es el registro seleccionado actualmente en el combo:



Consulta Utilizando sqlDataAdapter

```
using System;
using System.Windows.Forms;
using System.Data;
using System.Data.SqlClient;

namespace Acceso_datos
{
    public partial class frmLlenar_combogrid : Form
    {
        public frmLlenar_combogrid()
        {
            InitializeComponent();
        }

        SqlConnection conexion; // se define la variable para la conexión de tipo SqlConnection
        SqlDataAdapter da; // Definimos el adaptador de datos

        public void llenar_Datos()
        {
            dgClientes.Rows.Clear();

            // creamos la cadena de conexion con la base de datos
            conexion = new SqlConnection("Data Source=TECNOLOGIA2016;Initial Catalog=[DBFACTURAS];Integrated
Security=True");

            conexion.Open(); // invocamos método para abrir la base de datos

            try
            {
                string Sql = "Select * from tblclientes"; //Consulta a realizar
                da = new SqlDataAdapter(Sql, conexion); // Creamos el objeto da de tipo dataadapter
                DataTable tabla = new DataTable(); // creamos el objeto de tipo dataTable
            }
        }
    }
}
```




```
da.Fill(tabla); // llamamos la tabla con los datos consultados
dgClientes.DataSource = tabla; // asignamos el contenido de la tabla al grid

// llenamos el combo con el ID y con el nombre

cboClientes.DataSource = tabla; // asignamos la tabla al combo
cboClientes.DisplayMember = "StrNombre"; // indicamos campo a mostrar
cboClientes.ValueMember = "IdCliente"; // indicamos el id que se maneja en la base de datos
}
catch (Exception ex)
{
    MessageBox.Show("ERROR EN LA CONSULTA: " + ex.ToString());
}

conexion.Close(); // invocamos método para cerrar la base de datos
}

private void frmLlenar_combogrid_Load(object sender, EventArgs e)
{
    llenar_Datos();
}

private void btnInfo_Click(object sender, EventArgs e)
{
    string infocombo = $"Valor en el combo: \n ID:[ {cboClientes.SelectedValue} ] Nombre:[
{cboClientes.Text} ]";
    MessageBox.Show(infocombo);
}
}
```

Actualizando información en la base de datos

La sentencia **ExecuteNonQuery** se utiliza para ejecutar operaciones de manipulación de datos como UPDATE, INSERT o DELETE (SELECT no). **El valor devuelto corresponde al número de filas afectadas por la orden ejecutada.** También podemos utilizar este método para ejecutar operaciones de definición, por ejemplo, CREATE, ALTER o DROP

Ejemplo ingreso de registro con **ExecuteNonQuery**

```
string strMensaje = "";
conexion = new SqlConnection("Data Source=TECNOLOGIA2016;Initial Catalog=[DBFACTURAS];Integrated
Security=True");
conexion.Open(); // invocamos método para abrir la base de datos

if (txtNombre.Text != "")
{
    // ===== creamos variables con la informacion que vamos a actualizar =====
    string StrNombre = txtNombre.Text;
    double NumDocumento = 71684753;
    string StrDireccion = "Calle del espanto nro 1";
    string StrTelefono = "(456) 2345566";
    string StrEmail = "javier.saldarriaga@pascualbravo.edu.co";
    string DtmFechaModifica = DateTime.Now.ToShortDateString();
    string StrUsuarioModifica = "JAVIY2K";

    //===== dos formas de crear la sentencia para insertar en la base de datos, una con
    sentencia sql y otra con procedimiento =====
}
```




```
// string strConsulta = $"insert into TBLCLIENTES( StrNombre, NumDocumento, StrDireccion,
StrTelefono, StrEmail,DtmFechaModifica, StrUsuarioModifica) values('{StrNombre}', {NumDocumento},
'{StrDireccion}', '{StrTelefono}', '{StrEmail}', '{DtmFechaModifica}', '{StrUsuarioModifica}')";

// ***** NOTA, observe que el procedimiento pasamos como parametro un null, esto indica que
vamos a ingresar el registro
string strConsulta = $"Exec [actualizar_Cliente] null, '{StrNombre}', {NumDocumento},
'{StrDireccion}', '{StrTelefono}', '{StrEmail}', {StrUsuarioModifica}, '{DtmFechaModifica}'";

SqlCommand commando = new SqlCommand(strConsulta, conexion);
int retornado = commando.ExecuteNonQuery(); // UTILIZADO PARA UPDATE, INSERT y DELETE

conexion.Close(); // invocamos método para cerrar la conexion

if (retornado > 0) // retorna el nro de filas afectadas, si es cero no inserto datos
{
    strMensaje = "Los datos fueron Insertados";
}
else
{
    strMensaje = "Los datos no fueron Insertados";
}
MessageBox.Show(strMensaje);

llenar_grid(); // cargamos de nuevo el grid con datos para ver lo insertado
}
else {
    MessageBox.Show("Debe ingresar el nombre");
}
```

El siguiente link te permitirá descargar el proyecto de ejemplo en donde aplicamos los códigos anteriormente descritos para actualización de información, descárgalo para ejecutarlo y ver su funcionalidad y su código.

<https://drive.google.com/file/d/1HujkPmFfpFgF3R7lejM3PhQxpM8WGfOy/view?usp=sharing>

Creación de la clase “Acceso_datos”

Para facilitar el acceso a los datos de nuestra aplicación de “Proyecto_Sistema_Facturacion” crearemos en nuestro proyecto de visual studio una clase, esta clase contendrá los diversos métodos o funcionalidades que nos permita la conexión a la base de datos y la realización de consultas o de comandos para ingresar, modificar o retirar información de nuestras tablas en la base de datos. los siguientes son los métodos que implementaremos en esta clase:

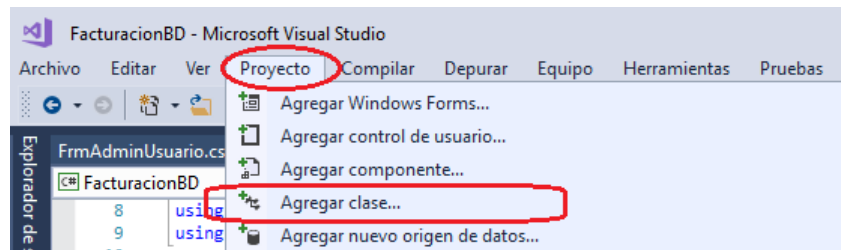
- **AbrirBd()** // método para abrir Conexión a la base de datos
- **CerrarrBd()** // método para cerrar conexión a la base de datos



- **ValidarUsuario**(string StrUsuario, string StrClave) // método para validar el ingreso del usuario al sistema
- **EjecutarComando**(string sentencia) // recibe una sentencia de para realizar acciones de actualizar, retirar y nuevo, solo retorna un valor numérico que indica cuantas filas fueron afectadas
- **cargartabla**(string tabla, string strCondicion) // Método que permite consultar una tabla y recuperar un conjunto de datos permite filtrar la información requerida
- **EjecutarComandoDatos**(string cmd) // Método que permite consultar con una sentencia (select) o invocar un procedimiento almacenado

Creando la clase “Acceso_datos”

Recordemos que la clase se crea accediendo en el menú “**Proyecto**” y luego seleccionamos agregar clase, a esta le daremos el nombre de “**Acceso_datos**” la siguiente imagen muestra las opciones de creación de la clase.



Para que la clase pueda funcionar adecuadamente, se debe anexar en la parte superior de la clase las siguientes librerías.

```
using System;  
using System.Data;  
using System.Data.SqlClient;  
using System.Windows.Forms;
```

Desarrollaremos en la clase un método que llamaremos “**AbrirBd()**”, este método utilizará el comando **SqlConnection** para establecer la conexión con la base de datos, así mismo creamos un método para cerrar la base de datos **CerrarrBd()**,



adicionalmente observe que colocamos la definición de los objetos de acceso a datos en la parte superior del código para que sea global a todos los métodos que implementaremos.

el siguiente es el código que se implementa en esta clase para Abrir y cerrar la conexión a la base de datos:

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms; // se utiliza inicialmente para mostrar los mensajes, a future no la
                             // utilizamos ya que la clase no debería mostrar los mensajes

namespace Sistema_facturacion
{
    class Acceso_datos
    {
        SqlConnection conexion; // se define la variable para la conexión de tipo SqlConnection
        SqlCommand cmd; // se define la variable para realizar comandos en la BD
        SqlDataReader LectorDatos = null;
        SqlDataReader dr;
        SqlDataAdapter da;
        DataTable dt;
        DataSet ds;

        public void AbrirBd() // método para abrir la base de datos
        {
            try // permite captura de un error en caso que se presente
            {
                // creamos un objeto de tipo conexión a la base de datos y se pasa como parámetro la cadena de conexión
                conexion = new SqlConnection("Data Source=TECNOLOGIA2016;Initial
Catalog=[DBFACTURAS];Integrated Security=True");
                conexion.Open(); // invocamos método para abrir la base de datos
            }
            catch (Exception ex) // si hay error presenta el siguiente mensaje
            {
                MessageBox.Show("falló conexión " + ex);
            }
        }

        public void CerrarBd() // método para cerrar la base de datos
        {
            try // permite captura de un error en caso que se presente
            {
                conexion.Close(); // invocamos método para cerrar la base de datos
            }
            catch (Exception ex) // si hay error presenta el siguiente mensaje
            {
                MessageBox.Show("falló cerrar conexión " + ex);
            }
        }
    }
}
```



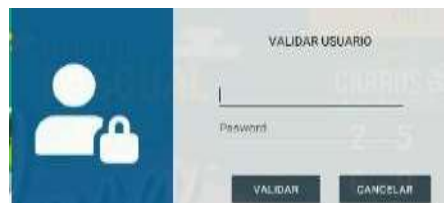
```
}  
}
```

Validar el usuario

En la parte inferior de la clase de la clase acceso a datos(`class Acceso_datos`) anexaremos un método llamado `ValidarUsuario`, el cual permite acceder a la base de datos para verificar si existe el usuario y la clave , el siguiente es el código a ingresar:

```
Clase validar Usuario  
  
// ===== Método para validar el ingreso del usuario al sistema =====  
public string ValidarUsuario(string StrUsuario, string StrClave)  
{  
    try  
    {  
        string strEmpleado = "";  
  
        string sentencia = $"select e.strNombre, e.IdRolEmpleado from TBLSEGURIDAD s JOIN TBLEMPLEADO  
e ON s.IdEmpleado = e.IdEmpleado where StrUsuario = '{StrUsuario}' and StrClave = '{StrClave}'";  
AbrirBd();  
cmd = new SqlCommand();  
// utilizamos las propiedades de SqlCommand esta es una forma extendidas con mas parámetros de  
control  
cmd.Connection = conexion;  
cmd.CommandText = sentencia;  
cmd.CommandType = CommandType.Text; // otros tipos son :CommandType.StoredProcedure  
CommandType.TableDirect  
cmd.CommandTimeout = 10;  
LectorDatos = cmd.ExecuteReader(); // ejecuta y retorna un conjunto de datos no actualizable  
while (LectorDatos.Read()) // recorremos los datos consultados  
    {  
        strEmpleado = Convert.ToString(LectorDatos.GetValue(0));  
    }  
    if (LectorDatos != null) // cerramos el lector de datos  
    {  
        LectorDatos.Close();  
    }  
  
    return strEmpleado;  
    }  
    catch (Exception ex)  
    {  
        MessageBox.Show("FALLA LECTURA: " + ex.Message);  
        return "";  
    }  
}
```

Para invocar este método abrimos nuestro formulario **FrmLogin**





Los nombres de los elementos principales son:

- Txtusuario
- Ttxtpassword (en propiedades colocar PasswordChar un asterisco *)
- btnValidar
- BtnCancelar

El siguiente es el código utilizado en el formulario para realizar la validación del usuario y la clave

```
frmLogin.cs [Diseño]
using System;
using System.Windows.Forms;

namespace Sistema_facturacion
{
    /// <summary>
    /// FUNCIONALIDAD: Por medio de este formulario realizamos el acceso a la base de datos con el fin de verificar el usuario y la clave
    /// POR: Javier Saldarriaga
    /// Fecha: Feb 2022
    /// </summary>
    /// <summary>
    /// </summary>
    public partial class FrmLogin : Form
    {
        /// <summary>
        /// </summary>
        public FrmLogin()
        {
            InitializeComponent();
        }

        /// <summary>
        /// </summary>
        private void btnCancelar_Click(object sender, EventArgs e)
        {
            Application.Exit();
        }

        /// <summary>
        /// </summary>
        private void btnValidar_Click(object sender, EventArgs e)
        {
            string Respuesta = ""; // creamos variable para controlar si encontró el usuario en la base de datos
            if (Txtusuario.Text != "" && txtpassword.Text != string.Empty) // verifico que usuario y clave no estén vacíos
            {
                Acceso_datos Acceso = new Acceso_datos(); // creamos un objeto con la clase Acceso_datos
                Respuesta = Acceso.validarUsuario(Txtusuario.Text, txtpassword.Text);

                if (Respuesta != "")
                {
                    MessageBox.Show("Bienvenido : " + Respuesta);
                    frmPrincipal frmppal = new frmPrincipal(); // creamos el objeto del formulario frmPrincipal
                    this.Hide(); // Ocultamos el formulario login
                    frmppal.Show(); // Mostramos el formulario principal
                }
                else
                {
                    MessageBox.Show("USUARIOS Y CLAVE NO ENCONTRADOS");
                    Txtusuario.Text = "";
                    Txtusuario.Focus();
                    txtpassword.Text = "";
                }
            }
            else
            {
                MessageBox.Show("debes ingresar un usuario y una clave");
            }
        }
    }
}
```

Procedemos a ejecutar el aplicativo y verificamos la validación del usuario desde la base de datos

Ahora vamos a completar nuestra clase de acceso a datos y le anexaremos los métodos que nos permitirán la consulta de información y la actualización de las misma.

Ejecutar comandos

El método EjecutarComando(string sentencia) recibe una sentencia de para realizar acciones de actualizar, retirar y nuevo, esta sentencia puede ser una instrucción



SQL o la invocación a un procedimiento almacenado, solo retorna un valor numérico que indica cuantas filas fueron afectadas,

```
83 public string EjecutarComando(string sentencia)
84 {
85     string salida = "LOS DATOS SE ACTUALIZARON SATISFACTORIAMENTE!";
86
87     try
88     {
89         int retornado;
90         AbrirBd();
91         cmd = new SqlCommand(sentencia, conexion);
92         retornado = cmd.ExecuteNonQuery(); // UTILIZADO PARA UPDATE, INSERT y DELETE
93         CerrarBd();
94         if (retornado > 0)
95         {
96             salida = "Los datos fueron Actualizados";
97         }
98         else
99         {
100             salida = "Los datos no fueron Actualizados";
101         }
102     }
103     catch (Exception ex)
104     {
105         salida = "Falló inserción: " + ex;
106     }
107     return salida;
108 }
```

Cargar tabla

El método cargartabla(string tabla, string strCondicion) permite consultar una tabla y recuperar un conjunto de datos, recibe como parametro strCondicion el cual permite filtrar la información de la tabla consultada. Retorna un datatable, este metodo generalmente lo utilizaremos para llenar los Combobox.

```
112 public DataTable cargartabla(string tabla, string strCondicion)
113 {
114     try
115     {
116         AbrirBd();
117         string Sql = "Select * from " + tabla + " " + strCondicion;
118         da = new SqlDataAdapter(Sql, conexion);
119         ds = new DataSet();
120         da.Fill(ds, tabla);
121         DataTable dt = new DataTable();
122         dt = ds.Tables[tabla];
123         CerrarBd();
124         return dt;
125     }
126     catch (Exception ex)
127     {
128         MessageBox.Show("ERROR EN LA CONSULTA: " + ex.ToString());
129         return null;
130     }
131 }
```

Ejecutar comandos

El método EjecutarComandoDatos(string cmd) permite consultar con una sentencia (select) o invocar un procedimiento almacenado para recuperar información y retorna un dataTable con lo consultado, lo utilizaremos generalmente para llenar los DataGridView.

```
134 public DataTable EjecutarComandoDatos(string cmd)
135 {
136     try
137     {
138         AbrirBd();
139         da = new SqlDataAdapter(cmd, conexion);
140         dt = new DataTable();
141         da.Fill(dt);
142         CerrarBd();
143         return dt;
144     }
145     catch (Exception ex)
146     {
147         MessageBox.Show("FALLO OPERACIÓN: " + ex);
148         return null;
149     }
150 }
```



Código completo de nuestra clase de acceso a datos

Acceso_datos.cs

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;

namespace Sistema_facturacion
{
    class Acceso_datos
    {
        SqlConnection conexion; // se define la variable para la conexión de tipo SqlConnection
        SqlCommand cmd; // se define la variable para realizar comandos en la BD
        SqlDataReader LectorDatos = null; // data reader , consulta de solo lectura de informacion
        SqlDataAdapter da;
        DataTable dt;
        DataSet ds;

        public void AbrirBd() // método para abrir la base de datos
        {
            try // permite captura de un error en caso que se presente
            {
                // creamos un objeto de tipo conexión a la base de datos y se pasa como parámetro la cadena de
                conexión
                conexion = new SqlConnection("Data Source=TECNOLOGIA2016;Initial
                Catalog=[DBFACTURAS];Integrated Security=True");
                conexion.Open(); // invocamos método para abrir la base de datos
            }
            catch (Exception ex) // si hay error presenta el siguiente mensaje
            {
                MessageBox.Show("falló conexión " + ex);
            }
        }

        public void CerrarBd() // método para cerrar conexión a la base de datos
        {
            try // permite captura de un error en caso que se presente
            {
                conexion.Close(); // invocamos método para abrir la base de datos
            }
            catch (Exception ex) // si hay error presenta el siguiente mensaje
            {
                MessageBox.Show("falló cerrar conexión " + ex);
            }
        }

        // ===== Método para validar el ingreso del usuario al sistema
        =====
        public string ValidarUsuario(string StrUsuario, string StrClave)
        {
            try
            {
                string strEmpleado = "";

                string sentencia = $"select e.strNombre, e.IdRolEmpleado from TBLSEGURIDAD s JOIN TBLEMPLEADO
                e ON s.IdEmpleado = e.IdEmpleado where StrUsuario = '{StrUsuario}' and StrClave = '{StrClave}'";
                AbrirBd();
                cmd = new SqlCommand();
                // utilizamos las propiedades de SqlCommand esta es una forma extendida con mas parametros de
                control
                cmd.Connection = conexion;
                cmd.CommandText = sentencia;
                cmd.CommandType = CommandType.Text; // otros tipos son :CommandType.StoredProcedure
                CommandType.TableDirect
                cmd.CommandTimeout = 10;
                LectorDatos = cmd.ExecuteReader(); // ejecuta y retorna un conjunto de datos no actualizable
                while (LectorDatos.Read()) // recorremos los datos consultados
```




```
        {
            strEmpleado = Convert.ToString(LectorDatos.GetValue(0));
        }
        if (LectorDatos != null) // cerramos el lector de datos
        {
            LectorDatos.Close();
        }

        return strEmpleado;
    }
    catch (Exception ex)
    {
        MessageBox.Show("FALLA LECTURA: " + ex.Message);
        return "";
    }
}

// ----- recibe una sentencia de para realizar acciones de actualizar, retirar y nuevo
// solo retorna un valor numerico que indica cuantas filas fueron afectadas
public string EjecutarComando(string sentencia)
{
    string salida = "LOS DATOS SE ACTUALIZARON SATISFACTORIAMENTE!";

    try
    {
        int retornado;
        AbrirBd();
        cmd = new SqlCommand(sentencia, conexion);
        retornado = cmd.ExecuteNonQuery(); // UTILIZADO PARA UPDATE, INSERT y DELETE
        CerrarBd();
        if (retornado > 0)
        {
            salida = "Los datos fueron Actualizados";
        }
        else
        {
            salida = "Los datos no fueron Actualizados";
        }
    }

    catch (Exception ex)
    {
        salida = "Falló inserción: " + ex;
    }
    return salida;
}

// Método que permite consultar una tabla y recuperar un conjunto de datos permite filtrar la
información requerida

public DataTable cargartabla(string tabla, string strCondicion)
{
    try
    {
        AbrirBd();
        string Sql = "Select * from " + tabla + " " + strCondicion;
        da = new SqlDataAdapter(Sql, conexion);
        ds = new DataSet();
        da.Fill(ds, tabla);
        DataTable dt = new DataTable();
        dt = ds.Tables[tabla];
        CerrarBd();
        return dt;
    }

    catch (Exception ex)
    {
        MessageBox.Show("ERROR EN LA CONSULTA: " + ex.ToString());
        return null;
    }
}

// Método que permite consultar con una sentencia (select) o invocar un procedimiento almacenado
public DataTable EjecutarComandoDatos(string cmd)
{

```



```
try
{
    AbrirBd();
    da = new SqlDataAdapter(cmd, conexion);
    dt = new DataTable();
    da.Fill(dt);
    CerrarBd();
    return dt;
}
catch (Exception ex)
{
    MessageBox.Show("FALLÓ OPERACIÓN: " + ex);
    return null;
}
}
}
```

Código para manejo del CRUD Clientes.

Teniendo ya la validación de usuario y password continuamos con la organización de los diferentes formularios que componen nuestra aplicación. Para esto desarrollaremos como guía la programación para el CRUD de clientes.

Programando formulario frmListaClientes

Realizaremos el código necesario para el formulario **frmListaClientes** en este colocaremos las funcionalidades de listar los clientes, buscar un nombre específico, llamado a ventana de nuevo y editar y finalmente implementaremos el borrado de registros. Partimos del formulario ya creado frmListaClientes:

Para que nuestro código para acceder a los datos funcione se requiere invocar al inicio del código del formulario la siguiente librería:

using System.Data



En el evento load del formulario invocamos la función **LLENAR_GRID()**, esta función se encarga de consultar en la base de datos todos los registros que se tienen en la tabla **tblClientes** y los muestra en el Datagridview llamado **dgClientes**. Para que la función pueda consultar en la base de datos necesitamos tener en la parte superior de nuestro código la creación de un objeto a partir de la clase y creamos un objeto de tipo **DataTable** para almacenar lo consultado, esto se puede apreciar en el siguiente código

```
1 using System;
2 using System.Data;
3 using System.Windows.Forms;
4 namespace Sistema_facturacion
5 {
6     //referencia
7     public partial class frmListaClientes : Form
8     {
9         //referencia
10        public frmListaClientes()
11        {
12            InitializeComponent();
13        }
14
15        //referencia
16        private void LLENAR_GRID()
17        {
18            dgClientes.Rows.Clear(); // limpiamos contenido del datagrid
19
20            string sentencia = $"SELECT idCliente, strNombre, NumDocumento, strTelefono FROM tblCLIENTES"; // sentencia a ejecutar
21            dt = Acceso.EjecutarComandoDatos(sentencia); // invocamos el metodo de EjecutarComandoDatos que fue creado en nuestra clase
22            // recorremos los datos consultados para llenar el grid
23            foreach (DataRow row in dt.Rows) {
24                dgClientes.Rows.Add(row[0], row[1], row[2], row[3]);
25            }
26        }
27
28        //referencia
29        private void frmListaClientes_Load(object sender, EventArgs e)
30        {
31            LLENAR_GRID();
32        }
33    }
34 }
```

Botón Buscar

Esta funcionalidad permite filtrar la información que se presenta en el Datagridview para buscar por un nombre específico, hacemos dobleclick en el botón buscar y anexamos el siguiente código:

```
67 //referencia
68 private void BtnBuscar_Click(object sender, EventArgs e)
69 {
70     if (txtBuscar.Text != "")
71     {
72         dgClientes.Rows.Clear(); // limpiamos contenido del datagrid
73         // creamos sentencia de consulta con un parametro del texto a buscar
74         string sentencia = $"select * from tblCLIENTES where strNombre like '{txtBuscar.Text}%'";
75         dt = Acceso.EjecutarComandoDatos(sentencia); // ejecutamos la consulta
76         foreach (DataRow row in dt.Rows) { dgClientes.Rows.Add(row[0], row[1], row[2], row[3]); } // llenamos el grid con lo consultado
77         txtBuscar.Text = "";
78     }
79     else
80     {
81         LLENAR_GRID();
82     }
83 }
84 }
```

Programamos los botones de actualización.

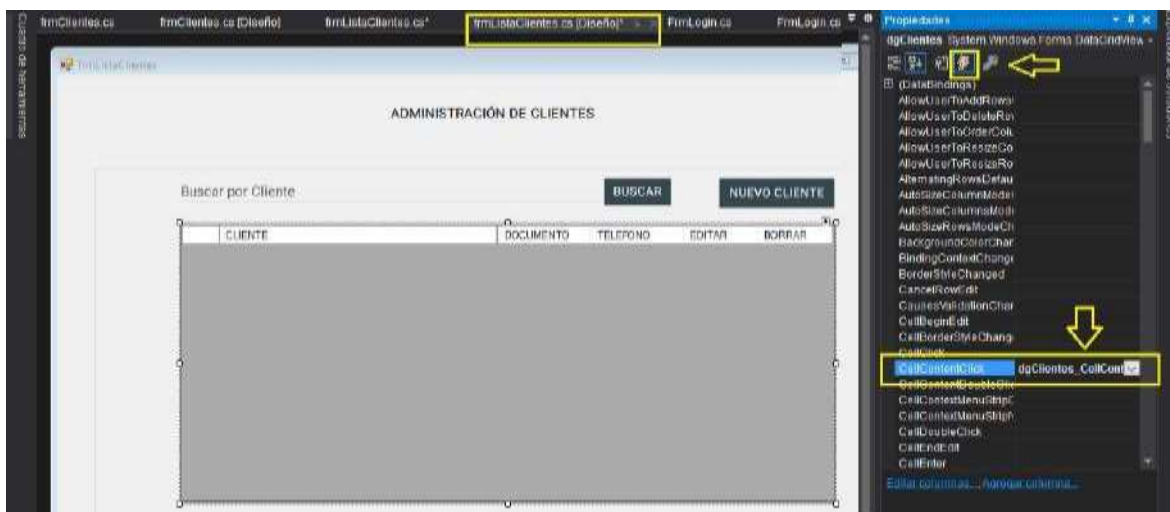
Botón nuevo, en este invocamos la ventana para ingresar un nuevo registro, el siguiente es el código:



```
15 // Referencia  
16 private void btnNuevo_Click(object sender, EventArgs e)  
17 {  
18     frmClientes cliente = new frmClientes(); // creamos el objeto de formulario  
19     cliente.IdCliente = 0; // este es una propiedad o atributo que debemos crear en el formulario frmClientes  
20     // esto permite pasar el IdCliente de este formulario frmClientes al formulario frmClientes  
21     // pasando el valor de cero lo indicamos que utilizaremos el formulario para ingresar un nuevo registro  
22     cliente.ShowDialog(); // mostramos el formulario en forma modal  
23     LLENAR_GRID(); // cuando sale de la ventana mostramos de nuevo el grid para ver el registro ingresado  
24 }  
25
```

Botones de editar y borrar del DataGridView

Crearemos el código que permite editar y borrar un registro particular del dataGrid, para esto tenemos que activar el evento “**dgClientes_CellContentClick**” esto lo realizamos seleccionando el datagrid, luego activamos la ventana de propiedades y en esta seleccionamos el icono “Rayo” el cual mostrara los eventos del datagrid y en la lista hacemos doble clic en **CellContentClick**:



En este nuevo evento creamos el siguiente código:

```
private void dgClientes_CellContentClick(object sender, DataGridViewCellEventArgs e)  
{  
    // verificamos si el botón presionado es el de borrar  
    if (dgClientes.Columns[e.ColumnIndex].Name == "btnBorrar")  
    {  
        int posActual = dgClientes.CurrentRow.Index; // Identificamos la fila a borrar  
        // mostramos un mensaje de confirmación de borrado  
        DialogResult resultado = MessageBox.Show("Seguro de borrar el cliente (" + dgClientes[posActual].Value.ToString() + ").", "CONFIRMACIÓN", MessageBoxButtons.YesNo, MessageBoxIcon.Question);  
        // si el resultado es No, no se borra el cliente  
        if (resultado == DialogResult.No)  
        {  
            return;  
        }  
        // si el resultado es Si, se borra el cliente  
        int IdCliente = Convert.ToInt32(dgClientes[posActual].Value.ToString()); // tomamos del datagrid el Id del cliente a borrar  
        string mensaje = "Se ha eliminado el cliente (" + IdCliente + ")";  
        string mensaje2 = "Acceso Ejecutar Comando (Verificar)";  
        MessageBox.Show(mensaje);  
        // Eliminar cliente  
        EliminarCliente(IdCliente);  
        LLENAR_GRID();  
    }  
    // verificamos si el botón presionado es el de editar  
    if (dgClientes.Columns[e.ColumnIndex].Name == "btnEditar")  
    {  
        int posActual = dgClientes.CurrentRow.Index;  
        frmClientes cliente = new frmClientes();  
        // pasamos el Id del cliente a editar y lo pasamos como parametro al siguiente formulario frmClientes  
        cliente.IdCliente = Convert.ToInt32(dgClientes[posActual].Value.ToString());  
        cliente.ShowDialog();  
        LLENAR_GRID();  
    }  
}
```

Finalmente colocamos en el btnSalir el código: **this.Close();**



Actualización de información en clientes

Recordemos que el siguiente formulario lo utilizaremos tanto para ingresar como para editar información del cliente, si recibe como parámetro en **IdCliente** un cero quiere decir que es un registro nuevo, pero si recibe un valor diferente significa que editaremos el registro que corresponde al **IdCliente** para actualizarlo.

En la parte superior del código colocamos el atributo **IdCliente** que permite recibir el parámetro del Id del cliente y adicionalmente creamos los objeto de acceso a los datos. Posteriormente hacemos doble clic en el formulario para crear el evento de carga del formulario (load) y en este invocamos el procedimiento **LLENAR_CLIENTE()**; y creamos el procedimiento.

El procedimiento básicamente verifica si el **idcliente** es cero para indicar que es un nuevo registro, pero si este tiene un valor indica que ya existe y que vamos a realizar su modificación, por lo cual procede a consultar la información en la base de datos, el siguiente es el código:

```
frmClientes.cs - x frmClientes.cs [Diseño] frmListaClientes.cs frmListaClientes.cs [Diseño] frmLogin.cs frmLogin.cs [Diseño]
ES Capa_presentacion
System.Windows.Forms
public frmClientes()
{
    InitializeComponent();
}

// Atributo que permite recibir como parámetro el idcliente
public int IdCliente { get; set; } // CREAMOS EL OBJETO DE TIPO DATATYPE PARA ALMACENAR LO CONSULTADO

// Procedimiento para llenar el cliente
private void LLENAR_CLIENTE()
{
    if (IdCliente == 0)
    {
        // ES UN REGISTRO NUEVO
        lblTitulo.Text = "INGRESO NUEVO CLIENTE";
    }
    else
    {
        // ACTUALIZAR EL REGISTRO CON EL ID PASADO
        string sentencia = "SELECT * FROM TB_CLIENTES WHERE idcliente = @idcliente"; // CONSULTA REGISTRO DEL IDCLIENTE
        dt = Acceso.EjecutaComandos(sentencia);
        foreach (DataRow row in dt.Rows)
        {
            // LLENAMOS LOS CAMPOS CON EL REGISTRO CONSULTADO
            txtNombre.Text = row[1].ToString();
            txtDocumento.Text = row[2].ToString();
            txtDireccion.Text = row[3].ToString();
            txtTelefono.Text = row[4].ToString();
            txtEmail.Text = row[5].ToString();
        }
    }
}

// Procedimiento para cargar el cliente
private void frmClientes_Load(object sender, EventArgs e)
{
    LLENAR_CLIENTE();
}
```



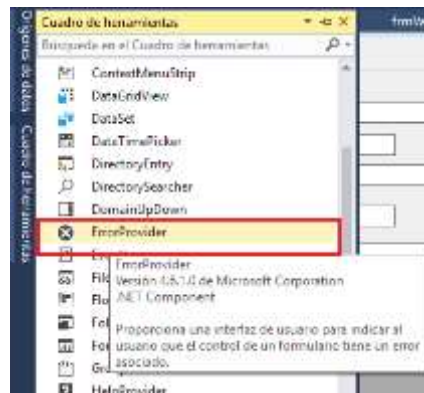

Boton guardar

Procedemos a realizar el código que permite almacenar el registro nuevo o el que estamos editando.

Hacemos doble clic en el botón `btnActualizar` y en este invocamos el procedimiento **Guardar()**; este primero realiza una validación de los campos para no dejar campos vacíos o valores no numéricos en campos que lo requieren, es por esto que se creó el procedimiento **Validar()**;

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000
```

Pero antes realizamos la funcionalidad que permite validar que la información de los campos sea correcta y no estén vacíos, para esto utilizamos el componente de la barra de herramientas llamada **errorprovider**,



Llevamos el componente **errorprovider** al formulario y le damos en propiedades el (name)= MensajeError

El siguiente código realiza el proceso de validar:



```
79
80 //FUNCIÓN QUE PERMITE VALIDAR LOS CAMPOS DEL FORMULARIO
81 //referencia
82 private Boolean validar()
83 {
84     Boolean errorCampos = true;
85     if (txtNombre.Text == string.Empty)
86     {
87         MensajeError.SetError(txtNombre, "debe ingresar el nombre del Cliente");
88         txtNombre.Focus();
89         errorCampos = false;
90     }
91     else { MensajeError.SetError(txtNombre, ""); }
92
93     if (TxtDocumento.Text == "")
94     {
95         MensajeError.SetError(TxtDocumento, "debe ingresar el documento");
96         TxtDocumento.Focus();
97         errorCampos = false;
98     }
99     else { MensajeError.SetError(TxtDocumento, ""); }
100
101     if (!esNumerico(TxtDocumento.Text))
102     {
103         MensajeError.SetError(TxtDocumento, "El Documento debe ser numerico");
104         TxtDocumento.Focus();
105         return false;
106     }
107     MensajeError.SetError(TxtDocumento, "");
108     return errorCampos;
109 }
110
111 //función para validar si un valor dado es numerico
112 //referencia
113 private bool esNumerico(string num)
114 {
115     try
116     {
117         double x = Convert.ToDouble(num);
118         return true;
119     }
120     catch (Exception)
121     {
122         return false;
123     }
124 }
```

Una vez validado los campos procedemos a guardar la información, es importante destacar que se invoca el procedimiento almacenado `Exec [actualizar_Cliente]` y que si el valor del parámetro `IdCliente` es cero, ingresa la información como nuevo, pero si tiene otro valor actualizará la información en la base de datos

Código completo formulario actualizar Cliente

```
using System;
using System.Data;
using System.Windows.Forms;
using MaterialSkin.Controls;

namespace Sistema_facturacion
{
    public partial class frmClientes : MaterialForm
    {
        public frmClientes()
        {
            InitializeComponent();
        }

        public int IdCliente { get; set; } // ATRIBUTO QUE PERMITE RECIBIR COMO PARAMETRO EL idCLIENTE

        DataTable dt = new DataTable(); // CREAMOS EL OBJETO DE TIPO DATATABLE PARA ALMACENAR LO CONSULTADO
        Acceso_datos Acceso = new Acceso_datos(); // creamos un objeto con la clase Acceso_datos

        private void LLENAR_CLIENTE()
        {
            if (IdCliente == 0)
            {
                // ES UN REGISTRO NUEVO
                lblTitulo.Text = "INGRESO NUEVO CLIENTE";
            }
            else
            {
            }
        }
    }
}
```




```
{
    //ACTUALIZAR EL REGISTRO CON EL ID PASADO
    string sentencia = $"select * from TBLCLIENTES where IdCliente = {IdCliente}"; // CONSULTO
    REGISTRO DEL idCLIENTE

    dt = Acceso.EjecutarComandoDatos(sentencia);
    foreach (DataRow row in dt.Rows)
    {
        // LLENAMOS LOS CAMPOS CON EL REGISTRO CONSULTADO

        txtNombre.Text = row[1].ToString();
        TxtDocumento.Text = row[2].ToString();
        txtDireccion.Text = row[3].ToString();
        txtTelefono.Text = row[4].ToString();
        txtEmail.Text = row[5].ToString();
    }
}

private void frmClientes_Load(object sender, EventArgs e)
{
    LLENAR_CLIENTE();
}

// ***** ACTUALIZACIONES *****

// ----- funciones que permiten el ingreso , retiro y actualización de la información de Clientes en
la base de datos
public bool Guardar()
{
    Boolean actualizado = false;

    if (validar())
    {
        try
        {
            Acceso_datos Acceso = new Acceso_datos();
            string sentencia = $"Exec [actualizar_Cliente] {IdCliente},{txtNombre.Text},{
TxtDocumento.Text} ,'{txtDireccion.Text}','{txtTelefono.Text}', '{txtEmail.Text}'
,'Javier',{DateTime.Now.ToShortDateString()}";
            MessageBox.Show(Acceso.EjecutarComando(sentencia));
            actualizado = true;
        }
        catch (Exception ex)
        {
            MessageBox.Show("falló inserción: " + ex);
            actualizado = false;
        }
    }
    return actualizado;
}

//FUNCIÓN QE PERMITE VALIDAR LOS CAMPOS DEL FORMULARIO
private Boolean validar()
{
    Boolean errorCampos = true;
    if (txtNombre.Text == string.Empty)
    {
        MensajeError.SetError(txtNombre, "debe ingresar el nombre del Cliente");
        txtNombre.Focus();
        errorCampos = false;
    }
    else { MensajeError.SetError(txtNombre, ""); }

    if (TxtDocumento.Text == "")
    {
        MensajeError.SetError(TxtDocumento, "debe ingresar el documento");
        TxtDocumento.Focus();
        errorCampos = false;
    }
    else { MensajeError.SetError(TxtDocumento, ""); }

    if (!esNumerico(TxtDocumento.Text))
    {
        MensajeError.SetError(TxtDocumento, "El Documento debe ser numerico");
    }
}
```



```
        TxtDocumento.Focus();
        return false;
    }
    MensajeError.SetError(TxtDocumento, "");
    return errorCampos;
}

//función para validar si un valor dado es numerico
private bool esNumerico(string num)
{
    try
    {
        double x = Convert.ToDouble(num);
        return true;
    }
    catch (Exception)
    {
        return false;
    }
}

private void btnActualizar_Click(object sender, EventArgs e)
{
    Guardar();
}

private void btn_Salir_Click(object sender, EventArgs e)
{
    this.Close();
}
}
```

Con lo visto al momento se ha detallado la funcionalidad y código necesario para la funcionalidad del CRUD de clientes, ahora usted deberá crear los siguientes CRUD de acuerdo a lo aprendido anteriormente

- Productos
- Categorías
- Facturas
- Empleados
- Roles

Administración de seguridad

Un formulario que No tendrá la funcionalidad utilizada en los CRUD es el de administración de la seguridad (**FrmAdminSeguridad**), este formulario tiene como fin el permitir el ingreso, retiro y actualización de los usuarios que pueden acceder al sistema, el formulario simplemente permite seleccionar un empleado y verifica si dispone de usuario y password , de tenerlo mostrará la información , en caso de no tenerlo permitirá su ingreso.

El siguiente es el formulario para este proceso.



Los campos definidos se detallan a continuación:

FrmAdminSeguridad	System.Windows.Forms.Form
btnActualizar	MaterialSkin.Controls.MaterialRaisedButton
BtnEliminar	MaterialSkin.Controls.MaterialRaisedButton
BtnNuevo	MaterialSkin.Controls.MaterialRaisedButton
BtnSalir	MaterialSkin.Controls.MaterialRaisedButton
cboEmpleado	System.Windows.Forms.ComboBox
FrmAdminSeguridad	System.Windows.Forms.Form
gpbDatosbasicos	System.Windows.Forms.GroupBox
lblEmpleado	System.Windows.Forms.Label
lblTitulo	MaterialSkin.Controls.MaterialLabel
TxtClave	MaterialSkin.Controls.MaterialSingleLineTextF

Se detalla el código implementado para el funcionamiento de esta pantalla:

```
using System;
using System.Data;
using System.Windows.Forms;

namespace FACTURACIONBD_2019
{
    public partial class FrmAdminSeguridad : Form
    {
        public FrmAdminSeguridad()
        {
            InitializeComponent();
        }

        private void llenar_combo_Empleados()
        {
            DataTable dt = new DataTable();
            Acceso_datos Acceso = new Acceso_datos(); // creamos un objeto con la clase Acceso_datos
            dt = Acceso.cargartabla("TBEMPLEADO", "");

            cboEmpleado.DataSource = dt;
            cboEmpleado.DisplayMember = "strNombre";
            cboEmpleado.ValueMember = "IdEmpleado";
            Acceso.CerrarrBd();
        }

        private Boolean validar()
        {
            Boolean errorCampos = true;
        }
    }
}
```



```
if (txtUsuario.Text == string.Empty)
{
    MensajeError.SetError(txtUsuario, "debe ingresar un valor de Usuario");
    txtUsuario.Focus();
    errorCampos = false;
}
else { MensajeError.SetError(txtUsuario, ""); }

if (Txtpassword.Text == "")
{
    MensajeError.SetError(Txtpassword, "Debe ingresar un valor de cédula");
    Txtpassword.Focus();
    errorCampos = false;
}
else { MensajeError.SetError(Txtpassword, ""); }

return errorCampos;
}

//metodo para validar si los valores son numericos
private bool IsNumeric(string num)
{
    try
    {
        double x = Convert.ToDouble(num);
        return true;
    }

    catch (Exception)
    {
        return false;
    }
}

// función que permite guardar los datos de ingreso a un usuario
public bool Guardar()
{
    Boolean actualizado = false;
    if (validar())
    {
        try
        {
            Acceso_datos Acceso = new Acceso_datos();
            string sentencia = $"Exec actualizar_Seguridad '{
Convert.ToInt32(cboEmpleado.Selected.Value)},{txtUsuario.Text}', '{Txtpassword.Text}', '{DateTime.Now}', 'Javier'";
            MessageBox.Show(Acceso.EjecutarComando(sentencia));
            actualizado = true;
        }
        catch (Exception ex)
        {
            MessageBox.Show("falló inserción: " + ex);
            actualizado = false;
        }
    }
    return actualizado;
}

// función que permite eliminar los datos de ingreso de un usuario
public void Eliminar()
{
    Acceso_datos Acceso = new Acceso_datos();
    string sentencia = $"Exec Eliminar_Seguridad '{ Convert.ToInt32(cboEmpleado.Selected.Value)}'";
    MessageBox.Show(Acceso.EjecutarComando(sentencia));
    txtUsuario.Text = "";
    Txtpassword.Text = "";
}

// función que permite consultar los datos de ingreso de un usuario
public void Consultar()
{
    DataTable dt = new DataTable();
    string sentencia = "select StrUsuario,StrClave from TBLSEGURIDAD where IdEmpleado=" +
cboEmpleado.Selected.Value.ToString();
```



```
Acceso_datos Acceso = new Acceso_datos(); // creamos un objeto con la clase Acceso_datos
dt = Acceso.EjecutarComandoDatos(sentencia);
if (dt.Rows.Count > 0)
{
    txtUsuario.Text = dt.Rows[0]["StrUsuario"].ToString();
    Txtpassword.Text = dt.Rows[0]["StrClave"].ToString();
}
else
{
    MessageBox.Show("El usuario no dispone de datos de ingreso");
    txtUsuario.Text = "";
    Txtpassword.Text = "";
}
}

private void FrmAdminSeguridad_Load(object sender, EventArgs e)
{
    llenar_combo_Empleados();
}

private void BtnGuardar_Click(object sender, EventArgs e)
{
    Guardar();
}

private void BtnEliminar_Click(object sender, EventArgs e)
{
    Eliminar();
}

private void BtnNuevo_Click(object sender, EventArgs e)
{
}

private void btnConsultar_Click(object sender, EventArgs e)
{
    Consultar();
}

private void BtnCancelar_Click(object sender, EventArgs e)
{
    //verificamos si desea cerrar la ventana
    DialogResult Rta;
    Rta = MessageBox.Show("Desea salir de la edición ?", "MENSAJE DE ADVERTENCIA",
    MessageBoxButtons.OKCancel, MessageBoxIcon.Warning);
    if (Rta == DialogResult.OK)
    {
        this.Close();
    }
}
}
```

Ya que se ha estudiado detalladamente esta guía proceda a las **Tareas Interactivas de Aprendizaje** y acceda a [“Tarea: Desarrollo de aplicación de escritorio cliente servidor con acceso a datos”](#) lea la GUIA PRACTICA DE LABORATORIO Nro 2 en la cual se le indicara los entregables que debe realizar en esta tarea.